Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07 Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА № 6

Дисциплина(модуль)/предмет МДК 02.02 Инструментальные средства разработки и отладки программного обеспечения (ПМ.02 Осуществление интеграции программных модулей)

Тема: Проектирование клиентской архитектуры и OpenAPI-спецификации интеграционного шлюза

ВАРИАНТ № 10

Студент (ка)/учащийся: Иванов Егор Михайлович / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: Чудинова А.А. /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «24» октября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

Введение....3
1 Разработка контракта API (Contract-First)....4
1.1 Эндпоинты интеграционного шлюза....4
1.2 Переиспользуемые модели данных....4
1.3 Проверка спецификации в Swagger Editor....5
2 Проектирование клиентской структуры....7
3 Реализация базового клиента....9
3.1 Класс-клиент SportsComplexApiClient....9
3.2 Модульные тесты....11
Заключение....14
Список использованных источников....15
Приложение А. Спецификация openapi.yaml....16

ВВЕДЕНИЕ

Цель работы — освоить методы контрактного проектирования API с использованием спецификации OpenAPI/Swagger и заложить слоистую архитектуру клиентского приложения (разделение UI, сетевого слоя и моделей) для разрабатываемой автоматизированной информационной системы (АИС).

Вариант задания определяется по номеру в списке группы. Номер — 10, поэтому по каталогу вариантов модулей АИС для практической работы № 6 выбран вариант 10: **АИС «Спортивный комплекс и секции»**. Модуль АИС — **интеграционный шлюз записи в секции, тренажёрный зал и аренду инвентаря**. Эндпоинты для спецификации по варианту: `GET /sections/schedule` (расписание спортивных секций) и `POST /equipment/rent` (бронирование инвентаря).

Задачи работы:

- спроектировать в Swagger Editor спецификацию OpenAPI версии 3.0.3 для двух взаимосвязанных эндпоинтов варианта с описанием переиспользуемых моделей в секции `components/schemas` и экспортировать контракт в файл `openapi.yaml`;
- создать в Visual Studio структуру клиентского модуля по принципу разделения ответственности (SoC): папки UI, Services, Models;
- написать класс-клиент `SportsComplexApiClient.cs`, инкапсулирующий логику запросов к шлюзу, и подготовить модульный тест проверки десериализации ответа;
- собрать решение, выполнить тесты и сделать выводы о преимуществах разделения интерфейсной части и сетевого слоя.

Использованное программное обеспечение: Swagger Editor 5.9.1 (editor.swagger.io) в браузере Google Chrome; Microsoft Visual Studio Community (версия 18); .NET SDK 8.0; тестовый фреймворк xUnit 2.9.3, Microsoft.NET.Test.Sdk 17.14.1. Вся работа выполнена на компьютере студента, скриншоты сделаны инструментом Windows «Ножницы».

1 РАЗРАБОТКА КОНТРАКТА API (CONTRACT-FIRST)

При подходе Contract-First сначала фиксируется контракт взаимодействия — спецификация OpenAPI, — и только потом по нему пишутся клиент и сервер. Контракт становится единым источником правды: клиентская команда может начинать работу, не дожидаясь готового сервера, а любые расхождения обнаруживаются на этапе проектирования.

1.1 Эндпоинты интеграционного шлюза

Два эндпоинта варианта связаны между собой: идентификатор занятия `sessionId`, который возвращает расписание, передаётся в запрос бронирования инвентаря — так студент бронирует мяч или наколенники к конкретной тренировке (таблица 1).

Таблица 1 – Эндпоинты спецификации openapi.yaml

Метод и путь	Назначение	Входные данные	Ответы
GET /sections/schedule operationId: getSectionsSchedule	Получение сетки занятий спортивных секций за период	query: dateFrom, dateTo (date, обязательные, период ≤ 14 дней), sectionType (enum), onlyAvailable (boolean)	200 — SectionSchedule; 400 — BadRequest (ApiError); 401 — Unauthorized
POST /equipment/rent operationId: rentEquipment	Бронирование спортивного инвентаря к занятию из расписания	body: EquipmentRentRequest (studentId, sessionId, items[1..5], quantity 1..3)	201 — EquipmentRentResponse; 400, 401; 404 — занятие не найдено; 409 — нет инвентаря на складе

Для обоих эндпоинтов задана схема безопасности `bearerAuth` (HTTP Bearer, JWT): шлюз принимает запросы только от авторизованных студентов. Общие ответы об ошибках вынесены в `components/responses` (BadRequest, Unauthorized) и переиспользуются в обеих операциях.

1.2 Переиспользуемые модели данных

Все структуры данных описаны один раз в секции `components/schemas` и подключаются в операции ссылками `$ref` (таблица 2). Это исключает дублирование: например, `RentItem` используется и в запросе, и в ответе бронирования, а `ApiError` — во всех ответах об ошибках.

Таблица 2 – Схемы components/schemas

Схема	Описание и ключевые ограничения
SectionType	Перечисление видов секций: VOLLEYBALL, BASKETBALL, FOOTBALL, SWIMMING, GYM, TABLE_TENNIS
SectionSession	Занятие секции: sessionId (шаблон ^SES-\d{4}-\d{4}-\d{2}$),

Схема	Описание и ключевые ограничения
	sectionType, title, coachName, hall, startsAt/endsAt (date-time), capacity (1..200), freeSlots (≥ 0), equipmentAvailable
SectionSchedule	Ответ GET /sections/schedule: dateFrom, dateTo, totalSessions, массив sessions[SectionSession]
RentItem	Позиция брони: equipmentCode (шаблон ^[A-Z]+(-[A-Z]+)*$), quantity (1..3)
EquipmentRentRequest	Тело POST /equipment/rent: studentId, sessionId, items (от 1 до 5 позиций RentItem)
RentStatus	Статус брони: CONFIRMED, WAITING_LIST, CANCELLED
EquipmentRentResponse	Ответ о брони: rentId, sessionId, status, pickupPoint, returnDeadline, items
ApiError	Единый формат ошибки: code, message, traceId (uuid)

1.3 Проверка спецификации в Swagger Editor

Спецификация набрана в редакторе Swagger Editor (editor.swagger.io). Редактор сразу строит интерактивную документацию: в правой панели отображаются название API, версия 1.0.0, метка OAS 3.0, сервер, кнопка авторизации Authorize, группы операций Schedule и Equipment и раздел Schemas (рисунок 1). Ошибок и предупреждений валидации нет.

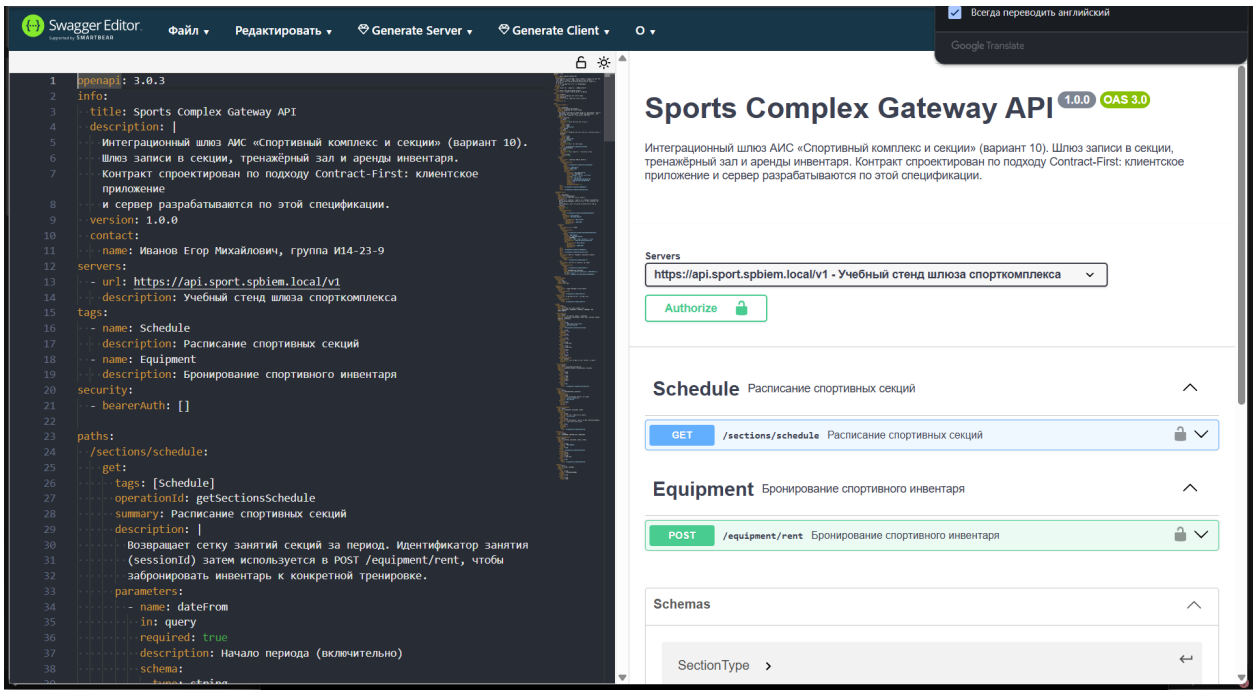


Рисунок 1 – Спецификация openapi.yaml в Swagger Editor: исходный код (слева) и сгенерированная документация (справа)

На рисунке 2 показана раскрытая операция `POST /equipment/rent`: Swagger Editor сформировал пример тела запроса по схеме `EquipmentRentRequest`, а в редакторе слева видна секция `components` со схемами `SectionType` и `SectionSession`.

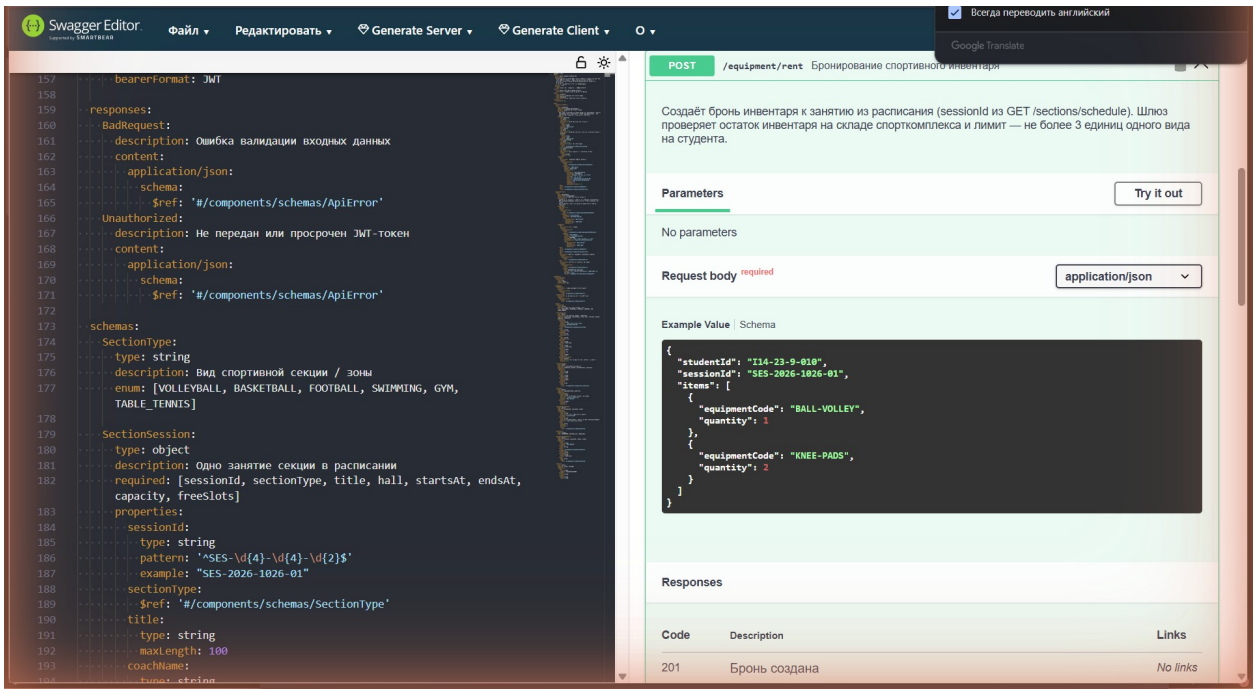


Рисунок 2 – Операция POST /equipment/rent и секция components/schemas в Swagger Editor

Готовый контракт сохранён в файл `openapi/openapi.yaml` решения (полный текст — в приложении А). Дополнительно файл проверен валидатором openapi-spec-validator — спецификация соответствует стандарту OpenAPI 3.0.3. Файл подключён к клиентскому проекту ссылкой (папка Contract), поэтому контракт всегда лежит рядом с кодом, который его реализует.

2 ПРОЕКТИРОВАНИЕ КЛИЕНТСКОЙ СТРУКТУРЫ

В Visual Studio создано решение `SportsComplexGateway` из двух проектов: консольное клиентское приложение `SportsComplex.Client` (.NET 8) и проект модульных тестов `SportsComplex.Client.Tests` (xUnit). Структура клиентского модуля построена по принципу разделения ответственности (Separation of Concerns, SoC) — таблица 3.

Таблица 3 – Слои клиентского модуля

Папка	Ответственность	Файлы
UI	Компоненты пользовательского интерфейса: меню, вывод расписания и результата брони. Работают только с интерфейсом ISportsComplexApiClient и ничего не знают об HTTP, URL и JSON	MainMenu.cs, ScheduleView.cs, RentEquipmentView.cs
Services	Сетевой слой: построение запросов к шлюзу, авторизация Bearer, сериализация, обработка кодов ошибок	ISportsComplexApiClient.cs, SportsComplexApiClient.cs, ApiJson.cs, SportsComplexApiException.cs
Models	Доменные модели — C#-аналоги схем components/schemas из openapi.yaml	SectionType, SectionSession, SectionSchedule, RentItem, EquipmentRentRequest, RentStatus, EquipmentRentResponse, ApiError
Contract	Ссылка на контракт openapi.yaml	openapi.yaml

Зависимости направлены строго в одну сторону: UI → Services → Models. Точка сборки зависимостей — `Program.cs`: в нём создаётся `HttpClient` с базовым адресом шлюза и передаётся в `SportsComplexApiClient`, а пользовательский интерфейс получает клиента через интерфейс. Структура каталогов решения приведена в листинге 1 и на рисунке 3.

Листинг 1 – Структура каталогов решения SportsComplexGateway

```
PR6_SportsComplex/
├── openapi/
│   └── openapi.yaml
├── src/
│   └── SportsComplex.Client/
│       ├── Models/
│       │   ├── ApiError.cs
│       │   ├── EquipmentRentRequest.cs
│       │   ├── EquipmentRentResponse.cs
│       │   ├── RentItem.cs
│       │   ├── RentStatus.cs
│       │   ├── SectionSchedule.cs
│       │   ├── SectionSession.cs
│       │   └── SectionType.cs
│       ├── Services/
│       │   ├── ApiJson.cs
│       │   ├── ISportsComplexApiClient.cs
│       │   ├── SportsComplexApiClient.cs
│       │   └── SportsComplexApiException.cs
```

```
│       ├── UI/
│       │   ├── MainMenu.cs
│       │   ├── RentEquipmentView.cs
│       │   └── ScheduleView.cs
│       ├── Program.cs
│       └── SportsComplex.Client.csproj
├── tests/
│   └── SportsComplex.Client.Tests/
│       ├── FakeHttpMessageHandler.cs
│       ├── SportsComplex.Client.Tests.csproj
│       └── SportsComplexApiClientTests.cs
└── SportsComplexGateway.sln
```

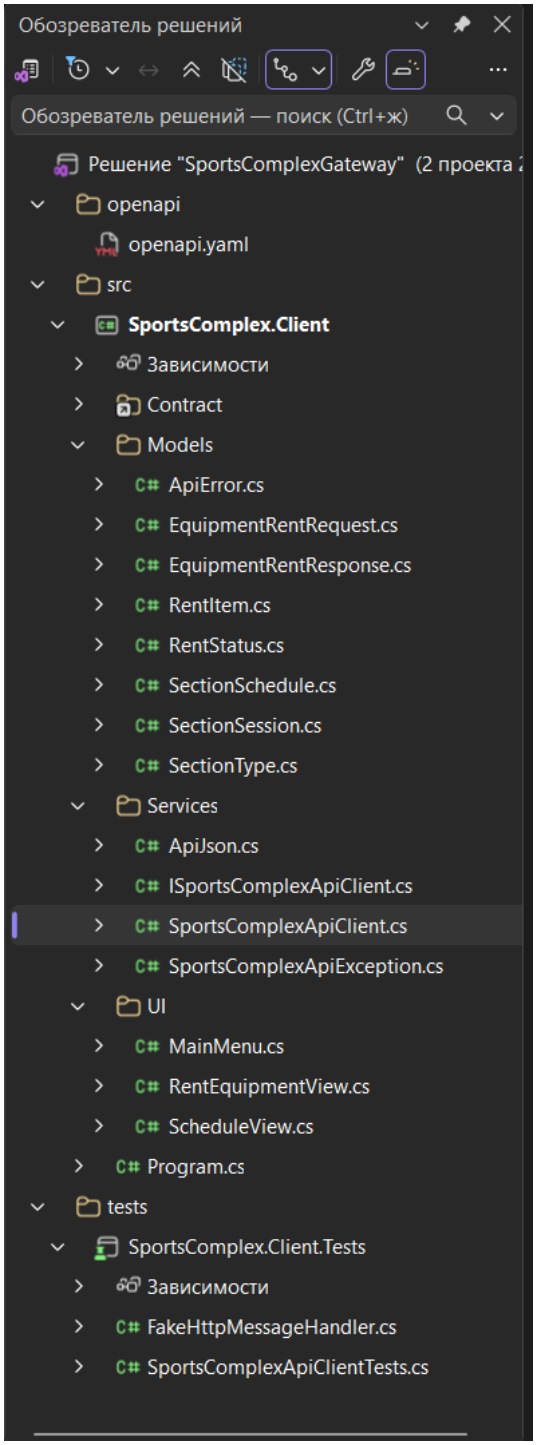


Рисунок 3 – Структура клиентского приложения в Обозревателе решений Visual Studio

3 РЕАЛИЗАЦИЯ БАЗОВОГО КЛИЕНТА

3.1 Класс-клиент SportsComplexApiClient

Класс `SportsComplexApiClient` реализует интерфейс `ISportsComplexApiClient` и инкапсулирует всю работу со шлюзом (листинг 2):

- строит URL запроса `sections/schedule` с параметрами dateFrom, dateTo, sectionType, onlyAvailable в формате, описанном в контракте;
- добавляет заголовок `Authorization: Bearer <JWT>`;
- проверяет входные данные до отправки запроса (период не больше 14 дней, количество одной позиции от 1 до 3) — те же ограничения, что в openapi.yaml;
- сериализует и десериализует JSON через общие настройки `ApiJson`: поля в camelCase, перечисления в UPPER_SNAKE_CASE (TABLE_TENNIS ↔ SectionType.TableTennis);
- превращает ответы 4xx/5xx в исключение `SportsComplexApiException` с кодом ошибки из схемы `ApiError`.

Листинг 2 – Класс-клиент SportsComplexApiClient.cs (папка Services)

```
using System.Net.Http.Headers;
using System.Net.Http.Json;
using System.Text.Json;
using SportsComplex.Client.Models;

namespace SportsComplex.Client.Services;

/// <summary>
/// Класс-клиент интеграционного шлюза спорткомплекса.
/// Инкапсулирует построение URL, авторизацию, сериализацию и обработку ошибок.
/// </summary>
public sealed class SportsComplexApiClient : ISportsComplexApiClient
{
    private const int MaxPeriodDays = 14;
    private readonly HttpClient _http;

    public SportsComplexApiClient(HttpClient http, string? accessToken = null)
    {
        _http = http ?? throw new ArgumentNullException(nameof(http));
        if (!string.IsNullOrWhiteSpace(accessToken))
        {
            _http.DefaultRequestHeaders.Authorization =
                new AuthenticationHeaderValue("Bearer", accessToken);
        }
    }

    public async Task<SectionSchedule> GetScheduleAsync(
        DateOnly dateFrom,
        DateOnly dateTo,
        SectionType? sectionType = null,
        bool onlyAvailable = false,
        CancellationToken cancellationToken = default)
    {
```

```
        if (dateTo < dateFrom)
            throw new ArgumentException("dateTo не может быть раньше dateFrom.");
        if (dateTo.DayNumber - dateFrom.DayNumber > MaxPeriodDays)
            throw new ArgumentException($"Период не может превышать {MaxPeriodDays}
дней.");

        var query = new List<string>
        {
            $"dateFrom={dateFrom:yyyy-MM-dd}",
            $"dateTo={dateTo:yyyy-MM-dd}"
        };
        if (sectionType is not null)
            query.Add($"sectionType={ToContractName(sectionType.Value)}");
        if (onlyAvailable)
            query.Add("onlyAvailable=true");

        using var response = await _http.GetAsync(
            "sections/schedule?" + string.Join('&', query), cancellationToken);

        return await ReadAsync<SectionSchedule>(response, cancellationToken);
    }

    public async Task<EquipmentRentResponse> RentEquipmentAsync(
        EquipmentRentRequest request,
        CancellationToken cancellationToken = default)
    {
        ArgumentNullException.ThrowIfNull(request);
        if (request.Items.Count == 0)
            throw new ArgumentException("Нужно выбрать хотя бы одну позицию инвентаря.");
        if (request.Items.Any(i => i.Quantity is < 1 or > 3))
            throw new ArgumentException("Количество одной позиции — от 1 до 3.");

        using var response = await _http.PostAsJsonAsync(
            "equipment/rent", request, ApiJson.Options, cancellationToken);

        return await ReadAsync<EquipmentRentResponse>(response, cancellationToken);
    }

    /// <summary>Перевод значения enum в имя из контракта (TableTennis →
TABLE_TENNIS).</summary>
    public static string ToContractName<TEnum>(TEnum value) where TEnum : struct, Enum =>
        JsonNamingPolicy.SnakeCaseUpper.ConvertName(value.ToString());

    private static async Task<T> ReadAsync<T>(HttpResponseMessage response,
CancellationToken ct)
    {
        if (!response.IsSuccessStatusCode)
        {
            ApiError? error = null;
            try
            {
                error = await response.Content.ReadFromJsonAsync<ApiError>(ApiJson.Options,
ct);
            }
            catch (JsonException)
            {
                // тело ошибки не соответствует контракту — оставляем только код
            }
            throw new SportsComplexApiException(response.StatusCode, error);
        }
```

```
        var body = await response.Content.ReadFromJsonAsync<T>(ApiJson.Options, ct);
        return body ?? throw new JsonException("Шлюз вернул пустой ответ.");
    }
}
```

Файл класса в редакторе Visual Studio показан на рисунке 4. В окне «Список ошибок» — 0 ошибок, 0 предупреждений, 0 сообщений.

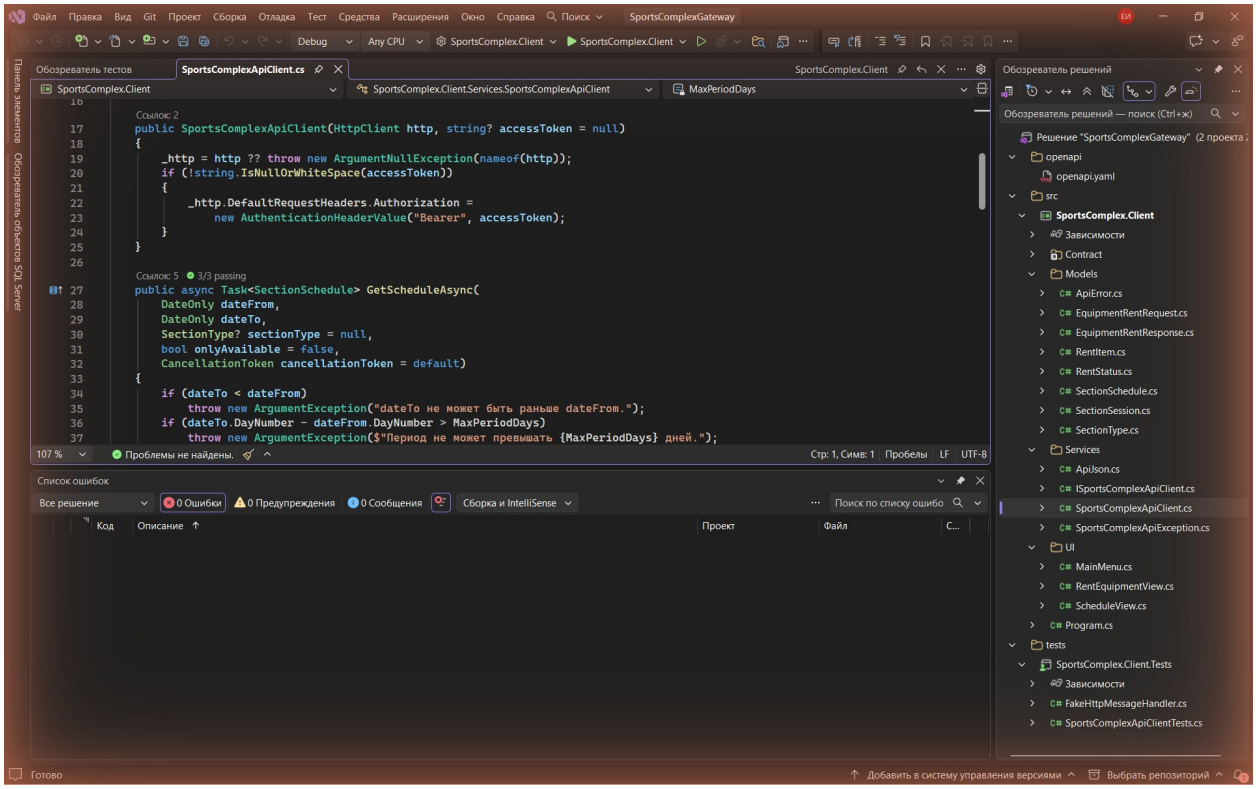


Рисунок 4 – Класс SportsComplexApiClient в Visual Studio, список ошибок пуст

3.2 Модульные тесты

Чтобы тесты не зависели от доступности реального шлюза, в тестовом проекте создан класс `FakeHttpMessageHandler`: он подменяет сеть, возвращает заранее заданный JSON и запоминает последний отправленный запрос. В качестве ответа используется пример из спецификации openapi.yaml. Подготовлено пять тестов (таблица 4), основной из них — проверка десериализации ответа (листинг 3).

Таблица 4 – Модульные тесты SportsComplexApiClientTests

Тест	Что проверяет
GetScheduleAsync_DeserializesResponseAccordingToContract	Десериализация ответа GET /sections/schedule: даты, DateTimeOffset с часовым поясом, перечисление "TABLE_TENNIS" → SectionType.TableTennis, необязательное поле coachName = null
GetScheduleAsync_BuildsQueryAndSendsBearerToken	Метод GET, итоговый URL с параметрами и заголовок Authorization: Bearer
RentEquipmentAsync_SerializesRequestAndDeserial	Сериализация тела POST /equipment/rent и разбор

Тест	Что проверяет
izesResponse	ответа 201 (status CONFIRMED)
RentEquipmentAsync_Conflict_ThrowsApiExceptionWithContractError	Ответ 409 превращается в SportsComplexApiException с кодом EQUIPMENT_NOT_AVAILABLE
GetScheduleAsync_PeriodLongerThan14Days_ThrowsBeforeSendingRequest	Проверка ограничения контракта (период ≤ 14 дней) до отправки запроса

Листинг 3 – Тест десериализации ответа (SportsComplexApiClientTests.cs)

```
    [Fact]
    public async Task GetScheduleAsync_DeserializesResponseAccordingToContract()
    {
        var (client, _) = Create(HttpStatusCode.OK, ScheduleJson);

        var schedule = await client.GetScheduleAsync(new DateOnly(2026, 10, 26), new
DateOnly(2026, 11, 1));

        Assert.Equal(new DateOnly(2026, 10, 26), schedule.DateFrom);
        Assert.Equal(2, schedule.TotalSessions);
        Assert.Equal(2, schedule.Sessions.Count);

        var volleyball = schedule.Sessions[0];
        Assert.Equal("SES-2026-1026-01", volleyball.SessionId);
        Assert.Equal(SectionType.Volleyball, volleyball.SectionType);
        Assert.Equal("Петров А.В.", volleyball.CoachName);
        Assert.Equal(new DateTimeOffset(2026, 10, 26, 18, 0, 0, TimeSpan.FromHours(3)),
volleyball.StartsAt);
        Assert.True(volleyball.HasFreeSlots);

        var tennis = schedule.Sessions[1];
        Assert.Equal(SectionType.TableTennis, tennis.SectionType);   // "TABLE_TENNIS" →
enum
        Assert.Null(tennis.CoachName);                               // необязательное поле
        Assert.False(tennis.HasFreeSlots);
        Assert.False(tennis.EquipmentAvailable);
    }
```

Решение пересобрано командой «Сборка → Пересобрать решение»: «Перестроить все: успешно — 2, неудачно — 0». Тесты запущены командой «Тест → Запуск всех тестов»: в Обозревателе тестов пройдено 5 из 5 тестов, не пройдено — 0 (рисунок 5).

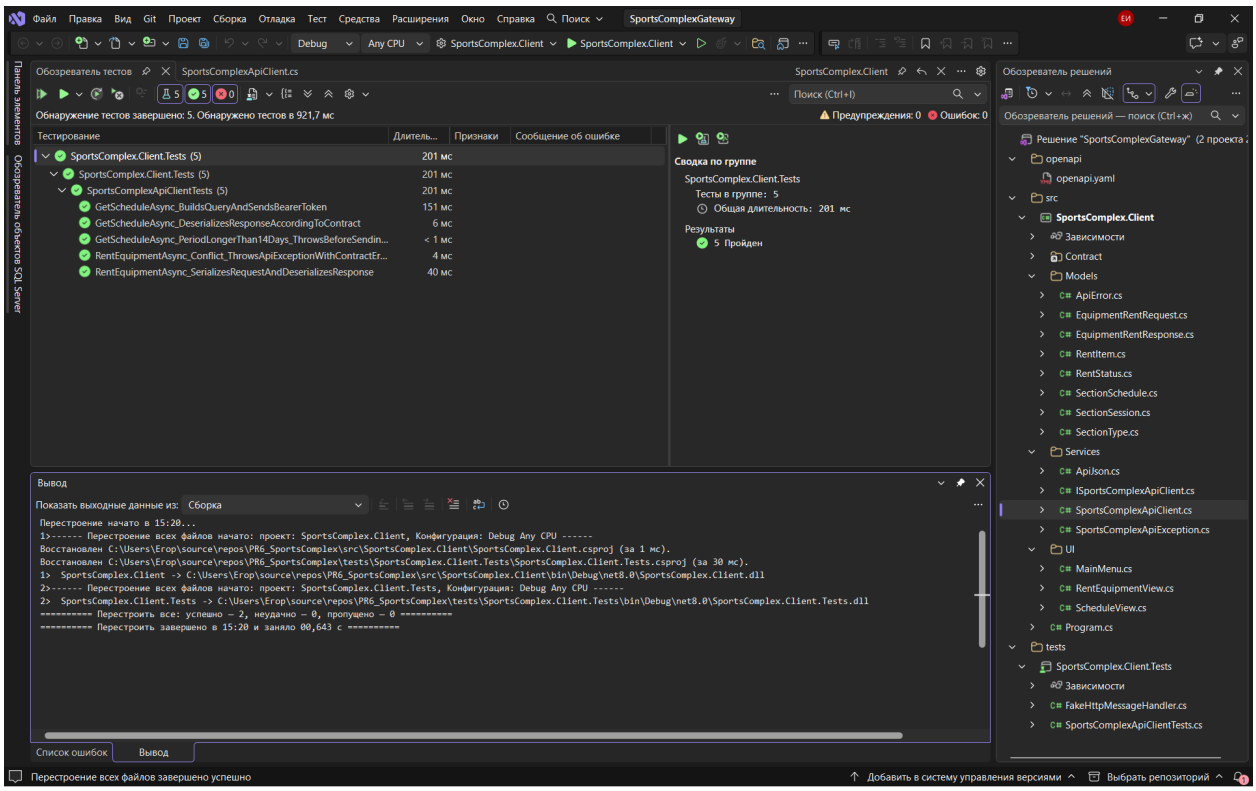


Рисунок 5 – Результаты: 5 тестов пройдено, пересборка решения выполнена успешно

ЗАКЛЮЧЕНИЕ

В ходе работы для АИС «Спортивный комплекс и секции» по подходу Contract-First спроектирована спецификация OpenAPI 3.0.3 интеграционного шлюза с двумя взаимосвязанными эндпоинтами `GET /sections/schedule` и `POST /equipment/rent`; восемь переиспользуемых моделей вынесены в `components/schemas`, общие ответы об ошибках — в `components/responses`. Спецификация проверена в Swagger Editor и сохранена в файл openapi.yaml. В Visual Studio создано клиентское приложение с разделением на папки UI, Services и Models, реализован класс-клиент `SportsComplexApiClient`, решение собирается без ошибок, все 5 модульных тестов проходят.

Выводы о преимуществах разделения интерфейсной части и сетевого слоя:

1) **Независимое изменение.** Если шлюз сменит адрес, формат авторизации или версию API, правится только слой Services; формы и меню UI остаются нетронутыми. И наоборот, консольный интерфейс можно заменить на WinForms или WPF без изменения сетевого кода.
2) **Тестируемость.** Сетевой слой проверяется модульными тестами с подменой HttpMessageHandler, без запуска реального сервера, а UI — с заглушкой интерфейса ISportsComplexApiClient.
3) **Переиспользование.** Один и тот же клиент шлюза можно подключить к нескольким интерфейсам (настольному, мобильному, служебной утилите).
4) **Соответствие контракту.** Модели Models являются прямым отражением схем openapi.yaml, поэтому изменения контракта сразу видны в одном месте и проверяются тестами десериализации.
5) **Параллельная работа команды.** Разработчик интерфейса и разработчик сетевого слоя работают одновременно, договорившись только об интерфейсе ISportsComplexApiClient и контракте OpenAPI.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) OpenAPI Specification v3.0.3 / OpenAPI Initiative. — URL: https://spec.openapis.org/oas/v3.0.3 (дата обращения: 24.10.2026).
2) Swagger Editor : онлайн-редактор спецификаций OpenAPI / SmartBear Software. — URL: https://editor.swagger.io (дата обращения: 24.10.2026).
3) Make HTTP requests with the HttpClient class / Microsoft Learn. — URL: https://learn.microsoft.com/dotnet/fundamentals/networking/http/httpclient (дата обращения: 24.10.2026).
4) System.Text.Json overview / Microsoft Learn. — URL: https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/overview (дата обращения: 24.10.2026).
5) Unit testing C# with xUnit and .NET / Microsoft Learn. — URL: https://learn.microsoft.com/dotnet/core/testing/unit-testing-with-dotnet-test (дата обращения: 24.10.2026).
6) Каталог вариантов модулей АИС для практической работы № 6 / ЭИОС СПбИЭУ, МДК 02.02. — 2026.

ПРИЛОЖЕНИЕ А. СПЕЦИФИКАЦИЯ OPENAPI.YAML

Листинг 4 – Полный текст openapi/openapi.yaml

```
openapi: 3.0.3
info:
  title: Sports Complex Gateway API
  description: |
    Интеграционный шлюз АИС «Спортивный комплекс и секции» (вариант 10).
    Шлюз записи в секции, тренажёрный зал и аренды инвентаря.
    Контракт спроектирован по подходу Contract-First: клиентское приложение
    и сервер разрабатываются по этой спецификации.
  version: 1.0.0
  contact:
    name: Иванов Егор Михайлович, группа И14-23-9
servers:
  - url: https://api.sport.spbiem.local/v1
    description: Учебный стенд шлюза спорткомплекса
tags:
  - name: Schedule
    description: Расписание спортивных секций
  - name: Equipment
    description: Бронирование спортивного инвентаря
security:
  - bearerAuth: []

paths:
  /sections/schedule:
    get:
      tags: [Schedule]
      operationId: getSectionsSchedule
      summary: Расписание спортивных секций
      description: |
        Возвращает сетку занятий секций за период. Идентификатор занятия
        (sessionId) затем используется в POST /equipment/rent, чтобы
        забронировать инвентарь к конкретной тренировке.
      parameters:
        - name: dateFrom
          in: query
          required: true
          description: Начало периода (включительно)
          schema:
            type: string
            format: date
          example: "2026-10-26"
        - name: dateTo
          in: query
          required: true
          description: Конец периода (включительно), не более 14 дней от dateFrom
          schema:
            type: string
            format: date
          example: "2026-11-01"
        - name: sectionType
          in: query
          required: false
          description: Фильтр по виду секции
          schema:
            $ref: '#/components/schemas/SectionType'
        - name: onlyAvailable
```

```
        in: query
        required: false
        description: Только занятия со свободными местами
        schema:
          type: boolean
          default: false
    responses:
      '200':
        description: Расписание успешно получено
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/SectionSchedule'
            example:
              dateFrom: "2026-10-26"
              dateTo: "2026-11-01"
              totalSessions: 1
              sessions:
                - sessionId: "SES-2026-1026-01"
                  sectionType: VOLLEYBALL
                  title: "Волейбол (сборная института)"
                  coachName: "Петров А.В."
                  hall: "Игровой зал №1"
                  startsAt: "2026-10-26T18:00:00+03:00"
                  endsAt: "2026-10-26T19:30:00+03:00"
                  capacity: 20
                  freeSlots: 6
                  equipmentAvailable: true
      '400':
        $ref: '#/components/responses/BadRequest'
      '401':
        $ref: '#/components/responses/Unauthorized'

/equipment/rent:
  post:
    tags: [Equipment]
    operationId: rentEquipment
    summary: Бронирование спортивного инвентаря
    description: |
      Создаёт бронь инвентаря к занятию из расписания (sessionId из
      GET /sections/schedule). Шлюз проверяет остаток инвентаря на складе
      спорткомплекса и лимит — не более 3 единиц одного вида на студента.
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/EquipmentRentRequest'
          example:
            studentId: "I14-23-9-010"
            sessionId: "SES-2026-1026-01"
            items:
              - equipmentCode: "BALL-VOLLEY"
                quantity: 1
              - equipmentCode: "KNEE-PADS"
                quantity: 2
    responses:
      '201':
        description: Бронь создана
        content:
```

```
          application/json:
            schema:
              $ref: '#/components/schemas/EquipmentRentResponse'
            example:
              rentId: "RENT-000452"
              sessionId: "SES-2026-1026-01"
              status: CONFIRMED
              pickupPoint: "Пункт выдачи инвентаря, 1 этаж"
              returnDeadline: "2026-10-26T20:00:00+03:00"
              items:
                - equipmentCode: "BALL-VOLLEY"
                  quantity: 1
                - equipmentCode: "KNEE-PADS"
                  quantity: 2
      '400':
        $ref: '#/components/responses/BadRequest'
      '401':
        $ref: '#/components/responses/Unauthorized'
      '404':
        description: Занятие с указанным sessionId не найдено
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ApiError'
      '409':
        description: Недостаточно инвентаря на складе
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ApiError'
            example:
              code: EQUIPMENT_NOT_AVAILABLE
              message: "Нет свободных наколенников (KNEE-PADS) на выбранное время"
              traceId: "c0a8012e-7f31-4b4e-9d1a-2f6c0e9a1b77"

components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

  responses:
    BadRequest:
      description: Ошибка валидации входных данных
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ApiError'
    Unauthorized:
      description: Не передан или просрочен JWT-токен
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ApiError'

  schemas:
    SectionType:
      type: string
      description: Вид спортивной секции / зоны
```

```
    enum: [VOLLEYBALL, BASKETBALL, FOOTBALL, SWIMMING, GYM, TABLE_TENNIS]

  SectionSession:
    type: object
    description: Одно занятие секции в расписании
    required: [sessionId, sectionType, title, hall, startsAt, endsAt, capacity, freeSlots]
    properties:
      sessionId:
        type: string
        pattern: '^SES-\d{4}-\d{4}-\d{2}$'
        example: "SES-2026-1026-01"
      sectionType:
        $ref: '#/components/schemas/SectionType'
      title:
        type: string
        maxLength: 100
      coachName:
        type: string
        maxLength: 80
      hall:
        type: string
        maxLength: 60
      startsAt:
        type: string
        format: date-time
      endsAt:
        type: string
        format: date-time
      capacity:
        type: integer
        minimum: 1
        maximum: 200
      freeSlots:
        type: integer
        minimum: 0
      equipmentAvailable:
        type: boolean
        description: Можно ли забронировать инвентарь к занятию
        default: true

  SectionSchedule:
    type: object
    description: Сетка занятий за период
    required: [dateFrom, dateTo, totalSessions, sessions]
    properties:
      dateFrom:
        type: string
        format: date
      dateTo:
        type: string
        format: date
      totalSessions:
        type: integer
        minimum: 0
      sessions:
        type: array
        items:
          $ref: '#/components/schemas/SectionSession'

  RentItem:
```

```
    type: object
    required: [equipmentCode, quantity]
    properties:
      equipmentCode:
        type: string
        description: Код позиции инвентаря на складе
        pattern: '^[A-Z]+(-[A-Z]+)*$'
        example: "BALL-VOLLEY"
      quantity:
        type: integer
        minimum: 1
        maximum: 3

  EquipmentRentRequest:
    type: object
    required: [studentId, sessionId, items]
    properties:
      studentId:
        type: string
        description: Номер студенческого билета
        example: "I14-23-9-010"
      sessionId:
        type: string
        description: Идентификатор занятия из GET /sections/schedule
        pattern: '^SES-\d{4}-\d{4}-\d{2}$'
      items:
        type: array
        minItems: 1
        maxItems: 5
        items:
          $ref: '#/components/schemas/RentItem'

  RentStatus:
    type: string
    enum: [CONFIRMED, WAITING_LIST, CANCELLED]

  EquipmentRentResponse:
    type: object
    required: [rentId, sessionId, status, items]
    properties:
      rentId:
        type: string
        example: "RENT-000452"
      sessionId:
        type: string
      status:
        $ref: '#/components/schemas/RentStatus'
      pickupPoint:
        type: string
      returnDeadline:
        type: string
        format: date-time
      items:
        type: array
        items:
          $ref: '#/components/schemas/RentItem'

  ApiError:
    type: object
    required: [code, message]
```

```
    properties:
      code:
        type: string
        example: VALIDATION_ERROR
      message:
        type: string
      traceId:
        type: string
        format: uuid
```